CUỘC THI KHOA HỌC KỸ THUẬT CẤP TRƯỜNG DÀNH CHO HỌC SINH TRUNG HỌC, NĂM HỌC 2026 – 2027

BÁO CÁO KẾT QUẢ NGHIÊN CỨU

Tên dự án:

MEMOLANG: NGHIÊN CỨU VÀ PHÁT TRIỂN CƠ CHẾ GIẢI PHÓNG VÙNG ĐỆM CHO PHÁT TRIỂN AI

Lĩnh vực: **Phần mềm hệ thống**

Hà Nội, tháng 9 năm 2026

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ	Mô tả
AI	Artificial Intelligence	Hệ thống máy tính thực hiện các tác vụ thường đòi hỏi trí tuệ con người
AOT	Ahead-Of-Time (compilation)	Biên dịch thành mã máy trước khi chương trình chạy
AST	Abstract Syntax Tree	Cây biểu diễn cấu trúc của mã nguồn
cuBLAS	CUDA Basic Linear Algebra Subprograms	Thư viện đại số tuyến tính tăng tốc bằng GPU của NVIDIA
CUDA	Compute Unified Device Architecture	Nền tảng tính toán song song của NVIDIA cho GPU
CUTLASS	CUDA Templates for Linear Algebra Subroutines	Thư viện mẫu C++ của NVIDIA cho đại số tuyến tính trên GPU
DRAM	Dynamic Random-Access Memory	Bộ nhớ chính cần được làm tươi định kỳ
FFI	Foreign Function Interface	Cơ chế gọi hàm được viết bằng ngôn ngữ khác
FP32	32-bit Floating Point	Định dạng số dấu phẩy động độ chính xác đơn
GEMM	General Matrix Multiplication	Quy trình chuẩn để nhân hai ma trận
GPU	Graphics Processing Unit	Bộ xử lý chuyên dụng cho tính toán song song cao
IR	Intermediate Representation	Dạng mã trung gian trong trình biên dịch, nằm giữa mã nguồn và mã máy
JIT	Just-In-Time (compilation)	Biên dịch thành mã máy trong khi chương trình đang chạy
LLM	Large Language Model	Mô hình AI được huấn luyện trên lượng lớn văn bản
LLVM	LLVM Compiler Infrastructure (originally “Low Level Virtual Machine”)	Bộ công cụ để xây dựng trình biên dịch và bộ sinh mã
MatMul	Matrix Multiplication	Phép nhân hai ma trận
REPL	Read–Eval–Print Loop	Vỏ tương tác thực thi từng dòng lệnh được nhập vào

Báo cáo kết quả nghiên cứu

Memo-Lang: Ngữ nghĩa đọc hủy trên các khối lượng công việc ma trận GPU

Dự án trình thông dịch độc lập
Tháng 9 năm 2026, Hà Nội

November 9, 2026

Abstract

Sự phát triển của AI trong thời gian gần đây đã dẫn đến sự bùng nổ của nhiều thuật toán máy học và mô hình học sâu. Do vậy, nhu cầu cho năng lực tính toán để học trên những bộ dữ liệu khổng lồ cũng từ đó tăng.

Hiện nay, phép toán được sử dụng rộng rãi cho thuật toán máy học là GEMM (General Matrix Multiplication - Phép nhân ma trận tổng quát). Trong quá trình học, mỗi phép nhân ma trận sẽ tạo ra một ma trận khác và ma trận mới sẽ được nối với những phép nhân ma trận khác. Để xử lý nhiều thao tác, tác vụ và quá trình khác nhau, những mô hình máy học cần thực hiện hàng triệu phép tính ma trận. Do những phép tính ma trận này so với các phép tính thông thường khác có tính đặc thù về thời gian rất cao, vì vậy những phép tính tiếp theo dần bị dồn lên bộ nhớ đệm và các bước trung gian cho đến khi những chương trình quản lý bộ nhớ hoặc quy tắc của máy tính giải phóng bộ nhớ.

Việc lặp lại quá trình này đối với mô hình máy học trở thành một vấn đề lớn trong việc hoạt động dưới một khối lượng công việc rất lớn, để lại hậu quả lớn cho hiệu suất và độ trễ của mô hình trên thiết bị và tài nguyên phần cứng nó đang sử dụng.

Bởi vậy, phần mềm đồng thời góp phần tối ưu hóa nhân của GEMM và tăng tuổi thọ của phần cứng được sử dụng, đảm bảo bộ đệm, dữ liệu đệm và con trỏ được giải phóng sớm nhất có thể và giảm phí tổn từ kết quả của việc dồn bộ nhớ đệm.

PART I: Câu hỏi nghiên cứu

Mục tiêu chính của chúng tôi là liên kết trực tiếp *thư viện chạy cuBLAS* đã biên dịch sẵn cho một ứng dụng cụ thể, được đánh giá bằng một phương pháp cụ thể, dưới một mô hình thực thi tương tự *hệ thống kiểu tuyến tính chặt chẽ*. Một số ngôn ngữ khác cũng theo đuổi ngữ nghĩa theo dõi việc tiêu thụ giá trị: hệ thống quyền sở hữu của Rust [**?**] áp đặt nguyên tắc dùng một lần tương tự thông qua hệ thống kiểu affine, còn Futhark [**?**] là ngôn ngữ mảng hàm thuần túy dùng kiểu duy nhất (uniqueness types) để cập nhật tại

chỗ một cách an toàn. Tuy nhiên, Futhark tự sinh kernel GPU từ mã nguồn thay vì tích hợp với các thư viện hiệu năng cao bên ngoài do bên thứ ba duy trì như cuBLAS, và đây được xem là một trong những trở ngại thực tế của ngôn ngữ này. Ngôn ngữ của chúng tôi thay vào đó tách việc phân tích tiêu thụ lúc biên dịch khỏi việc thực thi kernel.

$$T_1 = AB, \qquad T_2 = T_1C, \qquad T_3 = T_2D, \tag{1}$$

Trong triển khai thực tế, nhiều vùng đệm trung gian tồn tại đồng thời. Tốt nhất là vùng lưu trữ của các vùng đệm này được xóa và tái sử dụng ngay khi phép toán tương ứng đã hoàn tất hoặc khi vòng đời của đối tượng kết thúc.

Có thể kỳ vọng lợi ích về quản lý bộ nhớ từ một ngôn ngữ có ngữ nghĩa đọc hủy. Các phép toán số học như phép nhân ma trận có thể được giao cho kernel CUTLASS/cuBLAS đã tối ưu để vượt qua các hạn chế của Futhark [**?**]. Việc triển khai này được kỳ vọng không ảnh hưởng đến hiệu năng tính toán so với triển khai trên các ngôn ngữ khác. Tuy nhiên, cải thiện toàn trình được kỳ vọng xuất hiện nhờ các tối ưu hóa của ngôn ngữ, bao gồm:

- Giảm cấp phát tạm thời
- Rút ngắn vòng đời vùng đệm
- Giảm áp lực bộ nhớ GPU

Về lý thuyết, hiệu quả của cách triển khai này tăng theo khối lượng công việc, vì các tensor trung gian hoặc các phép toán ma trận nhỏ không chịu ảnh hưởng đáng kể từ việc quản lý bộ nhớ so với các chuỗi phép toán lớn.

Tuy nhiên, cần cân nhắc chi phí phát sinh vốn có, cũng như khả năng đánh giá quá cao chi phí bộ nhớ hoặc tính thực tiễn của việc tối ưu bộ nhớ. Do đó, dự án khảo sát các giả thuyết sau:

- Ngữ nghĩa đọc hủy có thể giảm mức sử dụng bộ nhớ đỉnh đối với các khối lượng công việc chứa các ma trận trung gian tồn tại ngắn.
- Đọc hủy có thể giảm số lần hoặc chi phí cấp phát vùng đệm tạm khi các vùng đệm đã tiêu thụ có thể được tái sử dụng một cách an toàn.
- Với các khối lượng công việc mà quản lý bộ nhớ chiếm phần đáng kể trong tổng thời gian thực thi, việc tái sử dụng vùng đệm có thể cải thiện hiệu năng toàn trình.

Các lợi ích thực tiễn tiềm năng gồm:

- Cải thiện hiệu năng của phép nhân ma trận, hỗ trợ huấn luyện và suy luận nhanh hơn cho các mô hình học máy.
- Tái sử dụng vùng đệm tensor trung gian hiệu quả hơn nhờ trình biên dịch.
- Giảm chi phí cấp phát vùng đệm và bộ nhớ đệm khi chạy.

I.1. Thuật toán

Phép nhân ma trận được viết như sau

$$C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} B_{k,j} \tag{2}$$

Mỗi giá trị của ma trận kết quả C được tạo ra bằng cách nhân một hàng của A với một cột của B rồi cộng các tích. Các triển khai hiệu quả phải quản lý việc di chuyển dữ liệu, tái sử dụng bộ nhớ và, trong sơ đồ đọc hủy, vòng đời của các vùng đệm toán hạng sau khi chúng bị tiêu thụ.

Đọc hủy là thao tác đọc bộ nhớ vốn xóa giá trị vừa đọc: dữ liệu bị hủy cùng lúc với khi được truy cập. Nếu một giá trị cần được giữ lại, phải có thao tác ghi lại khôi phục nó ngay lập tức, hoặc một vùng đệm/bộ nhớ đệm bên ngoài phải giữ một bản sao.

Các nghiên cứu liên quan về hệ thống kiểu tuyến tính có cách tiếp cận tương tự ở cấp độ ngôn ngữ, yêu cầu mỗi biến được dùng đúng một lần.

Về mặt lịch sử, đọc hủy phổ biến trong thiết kế phần cứng và mạch điện hơn là trong phần mềm. Dybdahl và cộng sự [**?**] đã nghiên cứu hành vi đọc hủy trong DRAM nhúng và phát hiện rằng việc bỏ bộ đệm ghi lại, thay vào đó dựa vào bộ nhớ đệm (đọc dữ liệu vào bộ nhớ đệm mà không khôi phục ngay về DRAM, và chỉ ghi lại khi dòng bộ nhớ đệm bị thay thế), mang lại lợi ích đo được về điện năng và hiệu năng.

Dù kết quả này riêng cho hệ thống bộ nhớ phần cứng, nó gợi ra câu hỏi rộng hơn mà dự án nghiên cứu: liệu một nguyên tắc tiêu thụ-khi-đọc tương tự, áp dụng ở cấp phần mềm/trình thông dịch cho các khối lượng công việc nặng về GEMM, có thể mang lại kết quả tương tự hay không?

PART II: Thiết kế và phương pháp

Mục tiêu của ngôn ngữ là xác định liệu ngữ nghĩa đọc hủy ở cấp ngôn ngữ có thể mang lại lợi ích quản lý bộ nhớ đo được cho các chương trình GPU thiên về ma trận, đồng thời vẫn giữ được hiệu năng của các kernel nhân ma trận đã tối ưu hay không.

Dự án sẽ được triển khai và đánh giá thông qua các mục tiêu sau:

- Định nghĩa cú pháp lõi giống Python và ngữ nghĩa đọc hủy của ngôn ngữ.
- Xây dựng bộ phân tích từ vựng, bộ phân tích cú pháp, AST và quy trình sinh mã dựa trên LLVM.
- Sinh LLVM IR cho các phép toán số học và ma trận.
- Tích hợp phép nhân ma trận GPU đã tối ưu thông qua CuBLAS.
- Xây dựng cơ chế theo dõi quyền sở hữu và sự tiêu thụ tường minh cho các toán hạng ma trận.
- Xác định khi nào các vùng đệm ma trận đã tiêu thụ có thể được tái sử dụng một cách an toàn.

- Kiểm chứng kết quả sinh ra với một tham chiếu số học độc lập.
- Đo mức sử dụng bộ nhớ GPU đỉnh và hành vi cấp phát vùng đệm tạm.
- Đo cả thời gian thực thi kernel GPU và thời gian chạy toàn chương trình.
- So sánh thực thi đọc hủy với một triển khai tương đương không đọc hủy.
- So sánh ngôn ngữ với các mốc cơ sở là thư viện GPU đã tối ưu khi phù hợp.
- Lưu giữ nhật ký benchmark, kết quả kiểm tra tính đúng đắn, đầu ra của trình biên dịch và các sản phẩm build để kết quả thực nghiệm có thể tái lập.

Đánh giá sẽ không mặc định rằng đọc hủy mang lại cải thiện hiệu năng. Việc giảm mức sử dụng bộ nhớ đỉnh mà không giảm thời gian chạy sẽ được coi là một kết quả có ý nghĩa; còn nếu quan sát thấy thời gian chạy được cải thiện, sẽ phân tích xem nó xuất phát từ việc giảm chi phí quản lý bộ nhớ hay từ thay đổi ở chính kernel nhân ma trận.

II.1. Ngữ nghĩa đọc hủy

Đọc hủy coi một giá trị là đã bị tiêu thụ khi một phép toán có quyền sử dụng độc quyền giá trị đó. Sau khi bị tiêu thụ, giá trị gốc không còn khả dụng cho các phép toán tiếp theo, trừ khi lập trình viên chủ động tạo bản sao hoặc bảo toàn nó bằng cách khác.

Với các phép toán ma trận, điều này tạo cơ hội tái sử dụng vùng lưu trữ. Ví dụ,

```
C = A @ B
D = C @ W

```

có thể cho phép vùng lưu trữ gắn với A hoặc C được tái sử dụng khi trình biên dịch xác định rằng giá trị tương ứng không còn nơi tiêu thụ nào nữa.

Mục đích của cơ chế này không phải là thay đổi phép tính toán học mà phép nhân ma trận thực hiện. Thay vào đó, nó thay đổi vòng đời của các vùng đệm gắn với phép tính. Nếu một vùng đệm có thể được tái sử dụng mà không cần sao chép nội dung, việc triển khai có thể giảm mức tiêu thụ bộ nhớ đỉnh và hoạt động cấp phát.

Việc tối ưu này vốn phụ thuộc vào khối lượng công việc. Một ma trận còn nhiều nơi tiêu thụ trong tương lai thì không thể bị hủy một cách an toàn sau lần dùng đầu tiên. Trong trường hợp đó, việc triển khai có thể cần sao chép, mượn hoặc một cơ chế khác để bảo toàn giá trị. Do đó, ngữ nghĩa đọc hủy có thể tạo ra sự đánh đổi giữa hiệu quả bộ nhớ và tính linh hoạt khi tái sử dụng giá trị.

Vì vậy, ngôn ngữ coi đọc hủy là một cơ chế sở hữu tường minh, thay vì giả định rằng việc tiêu thụ mọi toán hạng chắc chắn cải thiện hiệu năng.

II.2. Kiến trúc trình biên dịch và thư viện chạy

II.3. Thiết kế cú pháp

Chúng tôi chọn quy ước cú pháp giống Python cho ngôn ngữ lập trình:

- Chú thích dùng dấu thăng.
- Kiểu nguyên thủy: số nguyên, số thực, luận lý và chuỗi.
- Khai báo biến.
- Hàm dùng ’def’.
- extern cho liên kết runtime/FFI (giữa các ngôn ngữ).
- Độ ưu tiên toán tử.
- Cấu trúc điều khiển.

Tuy nhiên, vì tính khả thi khi triển khai và các quyết định thiết kế tiếp theo, chúng tôi điều chỉnh một số điểm của cú pháp Python gốc:

- độ ưu tiên đặc biệt của clone() để sao chép bộ nhớ
- dùng dấu ngoặc nhọn thay cho thụt lề để xác định phạm vi hàm

II.4. Sơ đồ Free-on-Consume và thư viện chạy cuBLAS

Sơ đồ được nêu trong [**?**] sẽ được sử dụng với triết lý thiết kế đã điều chỉnh để hoạt động ở cấp phần mềm.

Table 1. Các lựa chọn triển khai sơ đồ trong thiết kế.

Sơ đồ	**Triển khai**
Dữ liệu trước hết được đọc từ DRAM vào bộ nhớ đệm.	Không khác biệt.
Dữ liệu được ghi lại vào DRAM khi dòng bộ nhớ đệm bị thay thế, bất kể dữ liệu có bị sửa đổi hay không.	Các toán hạng được giải phóng vô điều kiện, với giả định bộ nhớ đệm luôn bị sửa đổi.
Tái sử dụng kiến trúc bộ nhớ đệm hiện có.	Dùng cấp phát theo thoát phạm vi để quản lý bộ nhớ đệm.
Che giấu độ trễ ghi lại bằng chồng lấp nhiều bank (xen kẽ bank).	Che giấu chi phí giải phóng bộ nhớ sau các lần khởi chạy kernel GPU.

Về việc liên kết cuBLAS đã biên dịch sẵn:

- cuBLAS nên được liên kết động.
- Trình điều khiển nên liên kết và đóng gói cùng các tệp nhị phân đã biên dịch của cuBLAS.
- Để thuận tiện, được phân phối dưới dạng một tệp đóng gói.

II.5. Đường ống xử lý

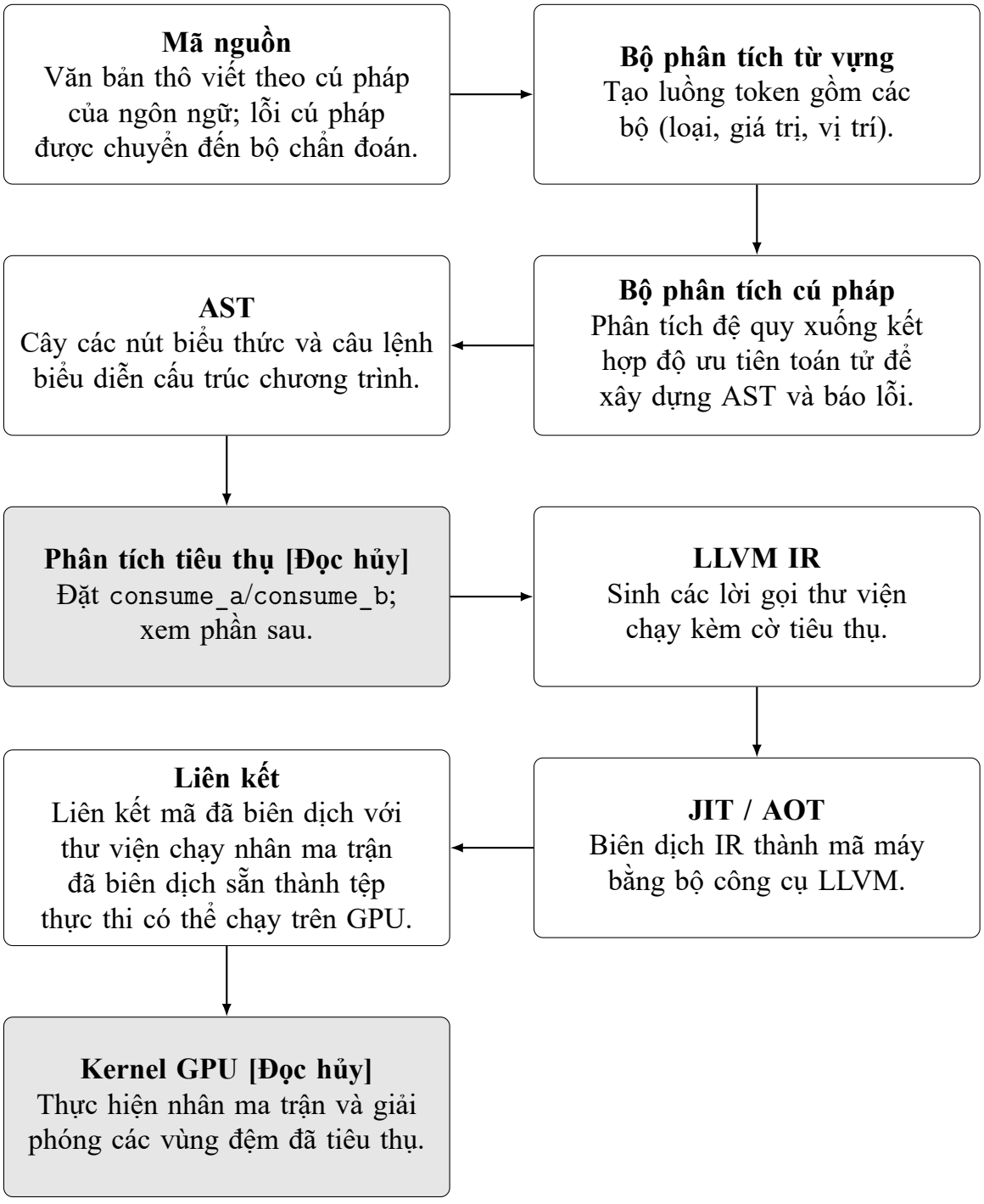


Figure 1. Đường ống biên dịch của ngôn ngữ.

Phần front-end của trình biên dịch được viết bằng C++; mỗi giai đoạn được thiết kế trước, sau đó lập trình, rồi gỡ lỗi độc lập để tìm lỗi, cuối cùng mới lắp ráp theo từng mô-đun.

PART III: Quá trình, xây dựng và kiểm thử

III.1. Quá trình

Table 2. Các tệp mã nguồn chính

Tệp	Mục đích
`Lexer.h/.cpp`	Tách token mã nguồn của ngôn ngữ
`Parser.h/.cpp`	Xây dựng AST
`ExprAST.h`	Định nghĩa các nút AST
`CodeGen.cpp`	Sinh LLVM IR
`CuBLASRuntime.cpp`	Liên kết (binding) MatMul của cuBLAS
`main.cpp`	Trình điều khiển / REPL

III.2. Kiểm thử

Table 3. Kết quả kiểm thử tính đúng đắn

Trường hợp kiểm thử	Kết quả
MatMul $N \times N$ so với tham chiếu (thông thường)	Chưa có
Phát hiện lỗi dùng lại giá trị đã đọc hủy (khi chạy)	Chưa có
Trường hợp biên (rỗng, 1x1, không vuông)	Chưa có

MatMul $N \times$N so với tham chiếu (thông thường): Đánh giá một chuỗi phép nhân ma trận để kiểm tra chịu tải nhằm đánh giá cải thiện toàn trình. Phát hiện lỗi dùng lại giá trị đã đọc hủy: Đánh giá khi chạy và khi biên dịch để phân loại lỗi. Trường hợp biên: Đánh giá các trường hợp biên để kiểm tra chịu tải về tính tương thích và độ toàn diện của thuật toán.

III.3. Đánh giá hiệu năng (Benchmark)

Việc đánh giá hiệu năng được thiết kế để phân biệt cải thiện về quản lý bộ nhớ với cải thiện của chính phép nhân ma trận.

Ba triển khai chính sẽ được so sánh:

1. Một triển khai thư viện GPU đã tối ưu, dùng làm tham chiếu hiệu năng kernel.
2. Ngôn ngữ dùng ngữ nghĩa giá trị thông thường (không đọc hủy).
3. Ngôn ngữ dùng ngữ nghĩa đọc hủy.

Khi phù hợp, một triển khai C++ hoặc CUDA đơn giản cũng sẽ được đưa vào làm mốc cơ sở.

Phép so sánh quan trọng nhất là giữa hai cấu hình của ngôn ngữ. Cả hai cấu hình thực hiện cùng các phép toán toán học và dùng cùng một triển khai nhân ma trận GPU bên dưới. Phép so sánh này cô lập ảnh hưởng của ngữ nghĩa đọc hủy tốt hơn so với việc so sánh với một triển khai không liên quan.

Bộ benchmark sẽ gồm các phép nhân ma trận đơn lẻ cũng như các chuỗi phép toán ma trận. Kích thước ma trận sẽ gồm nhỏ, vừa, lớn và không vuông. Điều này cho phép đánh giá xem ảnh hưởng của đọc hủy có phụ thuộc vào quy mô khối lượng công việc hay không.

Các phép đo sau sẽ được thu thập:

- Thời gian thực thi kernel GPU.
- Thời gian thực thi toàn trình.
- Mức tiêu thụ bộ nhớ GPU đỉnh.
- Số lần cấp phát vùng đệm tạm.
- Số lần tái sử dụng vùng đệm.
- Thời gian truyền từ máy chủ sang thiết bị và ngược lại, khi có áp dụng.
- Chi phí phát sinh của trình biên dịch và thư viện chạy.
- Sai số số học so với triển khai tham chiếu.
- Công suất tiêu thụ của GPU.

Các phép đo GPU sẽ dùng nhiều lần lặp và các vòng khởi động phù hợp. Chi phí khởi tạo CUDA [**?**] và biên dịch sẽ được tách khỏi các phép đo thực thi ở trạng thái ổn định khi có thể.

Benchmark sẽ báo cáo cả số đo tuyệt đối và mức thay đổi tương đối. Không có cải thiện hiệu năng nào được quy cho ngữ nghĩa đọc hủy trừ khi các số đo tương ứng ủng hộ cách diễn giải đó.

III.4. Mối quan hệ kỳ vọng giữa ngôn ngữ và hiệu năng MatMul

Thời gian thực thi kỳ vọng có thể được phân tách gần đúng như sau

$$T_{\text{total}} = T_{\text{language}} + T_{\text{memory}} + T_{\text{launch}} + T_{\text{kernel}}. \tag{3}$$

Ngữ nghĩa đọc hủy chủ yếu nhắm vào T_{memory}. Chúng không được kỳ vọng làm giảm đáng kể T_{kernel} khi dùng cùng một kernel GPU đã tối ưu.

Với một phép nhân ma trận lớn, T_{kernel} có thể chiếm phần lớn tổng thời gian thực thi, dẫn đến cải thiện toàn trình rất ít quan sát được. Tuy nhiên, với các khối lượng công việc chứa nhiều tensor trung gian hoặc các phép toán ma trận nhỏ, T_{memory} và các chi phí chạy khác có thể trở nên đáng kể hơn.

Việc đánh giá hiệu quả của ngôn ngữ chủ yếu dựa vào việc đo T_{memory}.

III.5. Bằng chứng và khả năng tái lập

Ví dụ biên dịch:

```
  def basicadd(int i, int k) {
    return i + k;
    }
  # Basic compilation of source code, addition only. No showcase of
   GEMMs or cuBLAS integration.

```

Listing 1. Ví dụ biên dịch

PART IV: Kết luận

IV.1. Tiến độ

Dự án đã hoàn thành một phần công việc đã lên kế hoạch, bao gồm bộ phân tích từ vựng, bộ phân tích cú pháp, AST và một trình điều khiển cơ bản. Thiết kế cho phần còn lại của đường ống đã được chốt, nhưng việc triển khai các tính năng ở phía sau của đường ống chưa bắt đầu do hạn chế về thời gian.

IV.2. Công việc tương lai

Các tính năng ở phía sau của đường ống, bao gồm việc tích hợp thư viện chạy nhân ma trận đã biên dịch sẵn, vẫn cần được triển khai. Các tính năng đang được lên kế hoạch như sau:

- Kiểu vector và ma trận
- Trình điều khiển chẩn đoán và bộ máy chẩn đoán
- Phân tích tiêu thụ tĩnh (kiểm tra đọc hủy) và xử lý các vùng đệm và giá trị trung gian của kernel GPU
- Liên kết với thư viện chạy nhân ma trận đã biên dịch sẵn để tạo tệp thực thi có khả năng khởi chạy kernel GPU
- Tách phần sinh mã khỏi AST để việc hạ cấp mã sạch hơn
- Các bước tối ưu hóa
- Hệ thống macro (khai báo liên kết, trừu tượng hóa mẫu chuỗi nhân ma trận)

IV.3. Ý nghĩa

Dự án này vừa là một thí nghiệm nhằm khám phá các cải tiến tiềm năng cho phép toán GEMM trong các thuật toán học máy và AI, bổ sung cho thư viện cuBLAS [**?**] vốn đã được tối ưu cao, vừa là một phản hồi trước một vấn đề rộng hơn của lĩnh vực này. Việc

huấn luyện mô hình hiện nay tiêu tốn lượng bộ nhớ khổng lồ, và nhu cầu từ hạ tầng AI đã vượt quá nguồn cung chip nhớ toàn cầu, góp phần khiến giá DRAM và NAND tăng kéo dài, gây áp lực tài chính lên các công ty CNTT, các tổ chức nghiên cứu khoa học và người tiêu dùng [**?**]. Các thuật toán tính toán GEMM đã được tối ưu đến *gần giới hạn lý thuyết* về thông lượng thô; nghiên cứu này thay vào đó nhắm vào hiệu quả bộ nhớ, dùng một hệ thống sở hữu mới để giảm dung lượng bộ nhớ cần thiết cho mỗi khối lượng công việc, như một đóng góp nhằm giảm bớt áp lực từ phía cầu vốn đang gây ra tình trạng thiếu hụt bộ nhớ hiện nay.

IV.4. Tính mới

1. Sản phẩm được xây dựng dựa trên những ý tưởng đã có như kiểu tuyến tính và quyền sở hữu. Tính mới của dự án nằm ở việc áp dụng các ý tưởng đó để giải phóng và quản lý vùng đệm GPU, bằng cơ chế đọc hủy được kiểm tra tĩnh khi biên dịch.
2. Dự án tiếp cận đề tài theo hướng vừa thử nghiệm với *hệ thống kiểu tuyến tính chặt chẽ* vừa để chế tạo.

References

[1] Haakon Dybdahl, Per Gunnar Kjeldsberg, Marius Grannæs, and Lasse Natvig, "Destructive-read in embedded DRAM, impact on power consumption," *Journal of Embedded Computing*, vol. 2, no. 1, pp. 83–96, 2006.

[2] NVIDIA, "cuBLAS Library." `https://developer.nvidia.com/cublas`

[3] LLVM Project, "The LLVM Compiler Infrastructure." `https://llvm.org`

[4] N. D. Matsakis and F. S. Klock, "The Rust language," in *Proc. ACM SIGAda Ada Letters*, 2014. `https://www.rust-lang.org`

[5] T. Henriksen, N. G. W. Serup, M. Elsman, F. Henglein, and C. E. Oancea, "Futhark: purely functional GPU-programming with nested parallelism and in-place array updates," in *Proc. 38th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2017. `https://futhark-lang.org`

[6] NVIDIA, "Mixed-precision training." `https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/index.html`

[7] NVIDIA, "Accuracy Considerations" `https://docs.nvidia.com/deeplearning/tensorrt/latest/inference-library/accuracy-considerations.html`,

[8] TrendForce, "Memory Price Outlook for 1Q26 Sharply Upgraded," `https://www.trendforce.com/presscenter/news/20260202-12911.html`,

[9] NVIDIA, "CUDA " `https://developer.nvidia.com/cuda/toolkit`